Is the Apple Ready to Pick? Apple Ripeness Classification with Machine Learning

Oskar Raftevold, Abdo Ghosn and Thomas Grotle, 30 October 2026

GitHub: https://github.com/h597236/dat158-assignment-2-apple-ripeness

Website: https://dat158-assignment-2-apple-ripeness.onrender.com/

1: DESCRIBE THE PROBLEM

SCOPE

The goal of the project is a website that takes a photo of an apple tree and tells whether an apple is ripe enough to be picked. The project is intended as a first step towards the perception system of an apple-picking robot. Such a robot must first find an apple and then decide whether to pick it. We focus on the second step: classifying a single apple as ripe or unripe.

Today, apples are mostly picked by hand. The picker judges ripeness from colour and size, and from the time in the season. Many orchards pick the same tree several times, so the picker has to judge every apple. Without machine learning, one could write a fixed rule such as «more than X % red pixels = ripe». Such a rule is fragile. Lighting, shadows, leaves in front of the apple and different cameras change the colours a lot, and the threshold has to be tuned by hand. Machine learning lets us learn the decision boundary from thousands of labelled apples while combining several colour features. That makes it a promising solution.

In this project an apple is defined as ripe when its surface is mainly red. This is a proxy, not a measurement of starch, sugar content or firmness. It’s best suited for finding red apple like Fuji, not green ones.

The intended users are developers of picking robots or camera-based systems for yield estimation. A simplified version could also help fruit growers get a quick estimate of how large a share of the apples in a field is ripe.

Business objective: Labour is the largest cost in apple harvesting. A robot that picks by itself can reduce this cost and make it feasible to harvest more often. However, picking an unripe apple is more costly than leaving a ripe one for another round: an unripe apple is lost, while a ripe apple can be picked the next time the robot passes. We therefore require high precision for «ripe» and

accept lower recall. The relevant business metrics would be the share of apples picked at the right time, losses from unripe apples, and cost per picked apple compared with manual picking.

The model is one component of a larger pipeline:

- Camera and object detector: finds each apple in the image. On the website, the user does this by clicking on the apple.
- Crop and features: cuts out a square around the apple and computes colour features (src/features.py).
- Classifier: outputs the probability that the apple is ripe, and a threshold turns it into a yes/no decision.
- Robot arm: picks the apples classified as ripe.

Changes in one component affect the others. The crops on the website must resemble the training crops, which have a 10 % margin around the apple. A detector that produces larger or smaller boxes would therefore shift the feature distribution. A different camera or different lighting changes the colours, which we tested specifically (see Section 3).

Resources: The project was carried out by three students over roughly one week. Everything ran on desktop with a GPU. The download for the dataset is about 14 GB and needs roughly twice that amount of free disk space while it is extracted. A full hyperparameter search takes a few minutes on a CPU, because the models work on small feature vectors rather than full images.

METRICS

Only about 14 % of the apples in the dataset are ripe. Accuracy is therefore a poor metric: a model that always answers «unripe» gets about 86 % accuracy without being useful. We used these metrics:

- **F1 for the class «ripe»:** the main metric for comparing models and in the hyperparameter search.
- **Precision for «ripe»:** the share of the apples the robot would pick that are actually ripe. This relates directly to losses from unripe apples. We set a target of at least 90 % precision when choosing the threshold.
- **Recall for «ripe»:** the share of ripe apples that are found. Low recall means apples are left for the next round, which is less serious.

- **Robustness:** F1 when the test images are altered with different lighting and colours, since the website receives images from other cameras than the dataset.
- **Latency:** time per prediction. A robot must assess many apples per second.

Minimum requirements for the project to be considered a success: the model must be clearly better than the baseline (F1 = 0), precision for «ripe» should be around 90 % on the test set, and the website must respond without noticeable delay.

2: DATA

We used the Fuji Ripeness and Size Dataset (Zhu et al., 2025), downloaded from Kaggle. It contains 4027 images of Fuji apple trees taken in orchards, annotated in VIA format (VGG Image Annotator). Each apple has a box or polygon and the attribute «Ripeness» with the value ripe or unripe. The dataset builds on, among others, PFuji-Size (Gené-Mola et al., 2021) and OpenAccess RGBD Apple. After removing apples smaller than 20 pixels, 18317 cropped apples remained. This is enough for models that work on colour features. We did not collect data of our own.

The ground-truth labels come from the dataset. According to the authors, they are based on colour and the date the photo was taken, not on measured ripeness. We checked the labels with src/check_labels.py (Figure 1). A typical ripe apple in the dataset is only about 40 % red, and almost no apples are more than 80 % red. Some red apples are labelled unripe, and some yellow-green apples are labelled ripe. Part of this is caused by leaves in front of the apple, overexposure and blurry crops. The labels are therefore noisy, which puts an upper limit on how good any model can become.



Figure 1:Top: the reddest apples labelled unripe. Bottom: the least red apples labelled ripe.

Privacy and ethics: The images show only trees and fruit, no people, so there are no privacy issues. The dataset is licensed GPL-3.0 on Kaggle, and PFuji-Size is licensed CC BY 4.0 for research and teaching. We therefore do not store the raw data in the repository but provide a

script (scripts/download_data.py) that downloads it. Each user needs their own Kaggle token, which is stored outside the repository.

Data preparation: src/prepare_crops.py crops every annotated apple with a 10 % margin and writes metadata.csv with the label and the photo each apple comes from. The dataset has its own train/val/test split, but tiles of the same photo, and the same scene photographed from different heights, can end up in different parts. That would give overly optimistic test results. We therefore merged all images and made our own split with StratifiedGroupKFold, grouped by photo and scene: 14651 apples for training and 3666 for testing (about 80/20), with roughly the same share of ripe apples in both.

Representation and feature engineering: Each crop is turned into a numeric vector in four different ways (src/features.py), so that we could compare them:

- color (30 features): hue histogram, share of red and green pixels, and mean and spread in HSV and RGB.
- hue (24 features): only features describing hue and chromaticity r/(r+g+b), without brightness and saturation.
- simple (6 features): share of red, yellow and green, red minus green, and mean red and green chromaticity.
- pixels (768 features): the image scaled down to 16×16 pixels, the same idea as the MNIST example in the course.

Before the colour features are computed, the outer 10 % of the crop is removed, since it often contains leaves and sky. Grey and dark pixels (low saturation or brightness) are ignored. Linear models and SVC get standardised features through StandardScaler in a pipeline, while tree-based models need no scaling. The classes are imbalanced, so all models use class_weight="balanced".

3: MODELING

All modelling is in src/train.py and follows the steps from the course notebooks: baseline, comparison with cross-validation, hyperparameter search, threshold selection and a single final test. Cross-validation uses StratifiedGroupKFold with 5 folds, so apples from the same photo are never in both the training and the validation fold.

Baseline: A DummyClassifier that always predicts the most frequent class (unripe) gets F1 = 0.00 for «ripe». A simple rule on the share of red pixels is the natural non-ML alternative. The labels were set by humans based on colour, but since they are noisy (Figure 1), not even a human looking only at the crop would reach a perfect score.

Model comparison: We tried logistic regression, SGDClassifier, random forest, SVC with an RBF kernel, and HistGradientBoostingClassifier with monotonic constraints on each of the four feature sets. Table 1 shows cross-validated F1 for «ripe» on the training set.

Tabell 1: Cross-validated F1 for «ripe» (5 folds, grouped by photo).

Model	**color**	**hue**	**simple**	**pixels**
Dummy (most frequent class)	0.00	0.00	0.00	0.00
Logistic regression	0.65	0.66	0.66	0.66
SGDClassifier	0.60	0.63	0.64	0.60
Random forest	0.77	0.75	0.71	0.68
SVC (RBF)	0.66	0.65	0.65	0.72
Gradient boosting (monotonic)	0.74	0.71	0.64	0.72

Random forest on the color features was best in cross-validation (F1 = 0.77). The pixel feature set was not better than the hand-crafted colour features, despite having 768 features. This supports that colour is the most important signal. Figure 2 shows which features random forest relied on most in an earlier run on the hue features: the share of red and the red bins of the hue histogram.

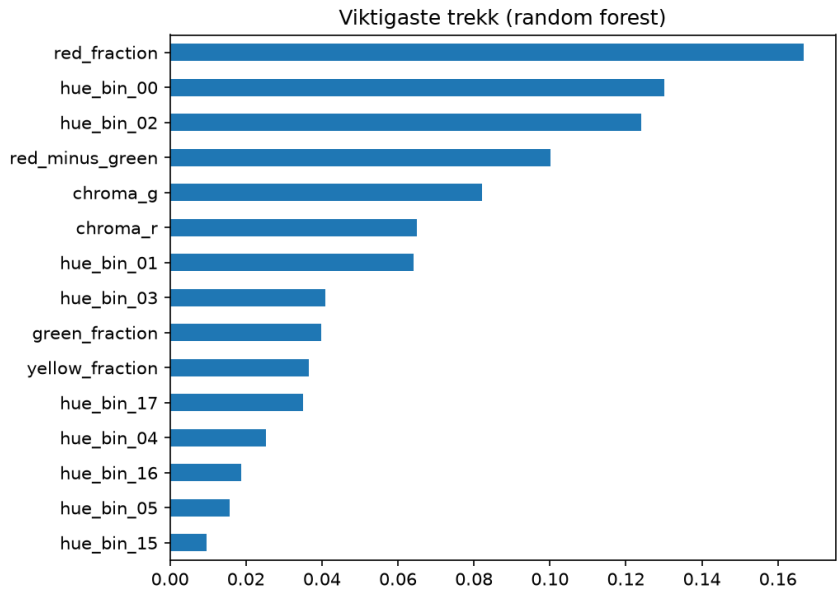


Figure 2: Most important features for random forest (hue features).

Robustness: When we tested the website on images from the internet, the model was uncertain, even for apples that were almost completely red. We therefore wrote src/robustness.py, which alters the test images the way a different camera or different lighting would, and measures F1 for random forest on each feature set (Table 2).

Tabell 2: F1 for «ripe» on the test set when the images are altered (random forest).

Model	**color**	**hue**	**simple**	**pixels**
Dummy (most frequent class)	0.00	0.00	0.00	0.00
Logistic regression	0.65	0.66	0.66	0.66
SGDClassifier	0.60	0.63	0.64	0.60
Random forest	0.77	0.75	0.71	0.68
SVC (RBF)	0.66	0.65	0.65	0.72
Gradient boosting (monotonic)	0.74	0.71	0.64	0.72

The pixel model dropped sharply (down to F1 = 0.15 for brighter images), while the hue features held up best on average. All feature sets struggled with cold light. We also found that random forest cannot extrapolate since almost no training apples are more than 80% red, the model does not know what to do with an apple that is redder than anything it has seen.

Final model: We therefore chose HistGradientBoostingClassifier with monotonic_cst on the six simple features. The constraints state that more red can never make an apple less ripe, and more green can never make it more ripe. With only six features, every feature can be given such a direction. The model is weaker than random forest in cross-validation (F1 = 0.64 versus 0.77), but behaves sensibly for apples outside the training data. We chose to trade some cross-validated F1 for a model that generalises better to the website, where images come from other cameras. GridSearchCV found learning_rate = 0.1, max_leaf_nodes = 63 and l2_regularization = 0.

Threshold selection: The threshold was chosen with precision_recall_curve on cross-validated probabilities from the training set, so the test set was not used for any choices. We chose the lowest threshold giving at least 90 % precision, which gave a threshold of 0.969 (Figure 3). We first tried 95 %, but then the model found only about a third of the ripe apples.

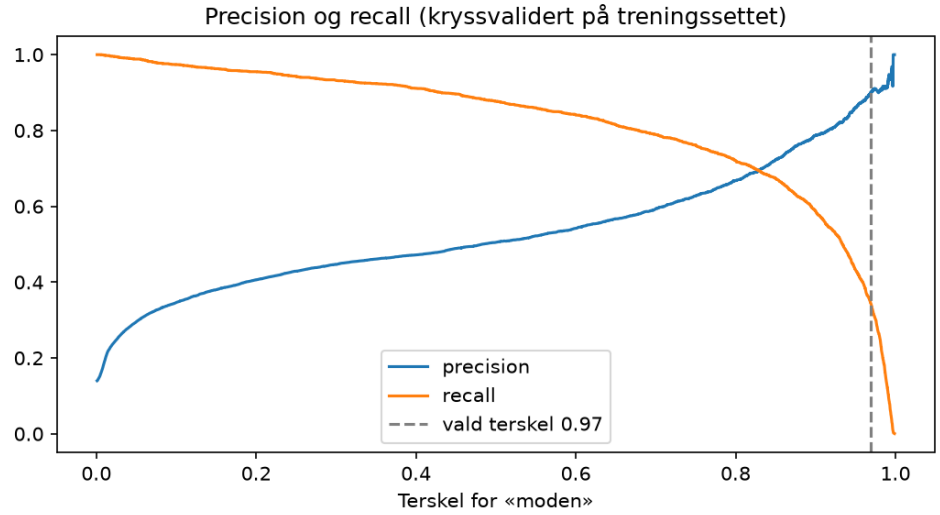


Figure 3: Precision and recall as a function of the threshold (cross-validated on the training set).

Final test: The model was evaluated once on the held-out test set. Table 3 shows the results at the default threshold of 0.5 and at the chosen threshold.

Tabell 3: Results on the test set (3666 apples, 516 ripe).

Metric (test set)	**Threshold 0.5**	**Threshold 0.97**
Precision «ripe»	0.52	0.86
Recall «ripe»	0.89	0.36
F1 «ripe»	0.65	0.51
Precision «unripe»	0.98	0.90

Recall «unripe»	0.86	0.99
Accuracy	0.87	0.90

At the chosen threshold, precision for «ripe» was 86%, slightly below the 90 % target that was reached in cross-validation. Recall was 36%: the model finds about one in three ripe apples, but of the 218 apples it calls ripe, only 31 are unripe (Figure 4). For a robot that makes several rounds this is the trade-off we wanted, but recall is lower than we hoped for. At a threshold of 0.5 the model finds 89 % of the ripe apples, but only half of the apples it calls ripe actually are.

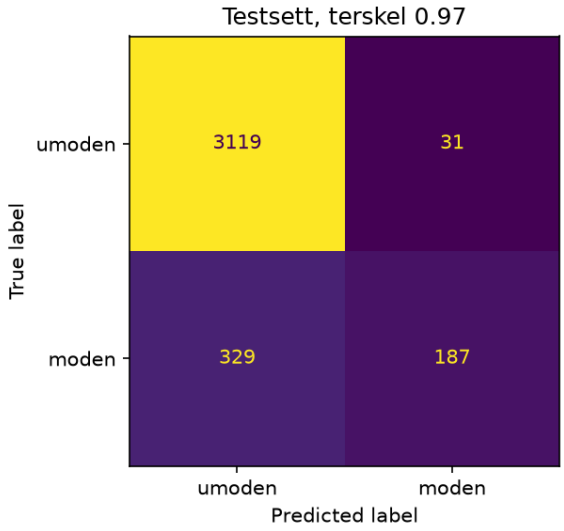


Figure 4: Confusion matrix on the test set at the chosen threshold (umoden = unripe, moden = ripe).

Error analysis: Figure 5 shows misclassified apples from the test set. Most are ripe apples with a pink or pale red colour in strong light, where the model gives probabilities between 0.6 and 0.95, i.e. below the threshold. Others are covered by leaves or branches, or the crop contains several apples. The errors show that the model has no notion of shape or surface and only sees colour. Possible improvements are more and more varied data, better labels based on measured ripeness, colour correction of the images before the features are computed, or a convolutional neural network that also sees shape and texture.

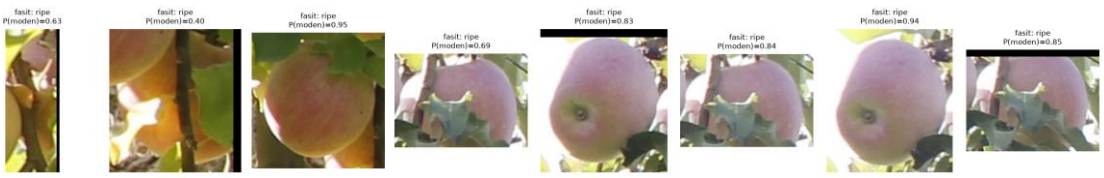


Figure 5: Misclassified apples in the test set, with true label («fasit») and probability of ripe.

4: DEPLOYMENT

The model is served through a website built with Gradio (app.py). The trained model, the feature set and the threshold are stored together in models/apple_model.joblib, and the website uses the same feature functions (src/features.py) as the training. This guarantees that the features are computed in exactly the same way.

The model is trained on crops where one apple fills the image. The user therefore uploads a photo of an apple tree and clicks on an apple. The app cuts out a square around the click with the same proportions as in training, shows the crop («Dette ser modellen» – «This is what the model sees»), and marks the apple with a red box for ripe and a green box for unripe. A table lists all assessed apples with their probability, and the user sees how many of them are ready to pick. A slider controls the crop size, and buttons let the user undo, clear all marks, or assess the whole image as a single close-up. One prediction takes about 5 ms on an ordinary laptop, so latency is not an issue.

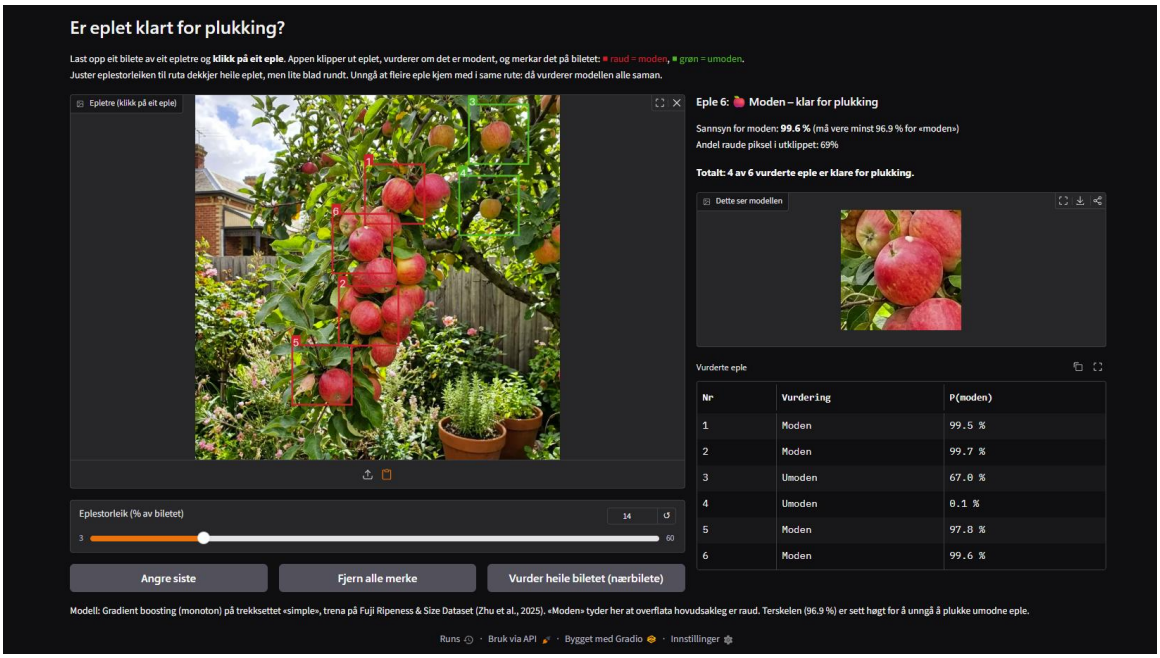


Figure 6: The website in use on a photo of a Gala apple tree from outside the dataset (photo: Nursery Near Me [9]). Four of six assessed apples are marked ripe (red box). Apple 3 is red but only gets 67 % and is treated as unripe, because the threshold is set high

Figure 6 also illustrates the limitations from Section 3. Some boxes cover several apples, and the model then assesses them together. Apple 3 is red with bright, sunlit areas and falls below the

threshold. This is the same type of error as in Figure 5, and a consequence of prioritising precision over recall.

Link to the website: https://dat158-assignment-2-apple-ripeness.onrender.com/

Source code and instructions for reproducing everything:
https://github.com/h597236/dat158-assignment-2-apple-ripeness

The website only needs app.py, src/features.py, models/apple_model.joblib and requirements.txt, so it can be deployed without the dataset.

Monitoring and maintenance: In production, the distribution of images will change with camera, lighting, cultivar and time of season. One should log the probabilities and the share of red the model sees, and watch for the distribution drifting away from the training data. In a robot, decisions should also be compared with a sample checked by a human. Precision must then be measured again and the threshold adjusted.

Future work: The most important next step is an object detector (deep learning, e.g. YOLO) that finds the apples automatically, so the user does not have to click. After that, the model should be trained on more cultivars and cameras, preferably with labels from measured ripeness (starch or sugar). Finally, it should be tested on images from a real orchard taken with the camera the robot will use.

Limitations: The model is trained on a single cultivar (Fuji) and does not suit green cultivars. The labels are based on colour and photo date, not measured ripeness. The user must find the apples and set the crop size manually.

5: REFERENCES

[1] Zhu, K. et al. (2025). Foundation Model-Based Apple Ripeness and Size Estimation for Selective Harvesting. arXiv:2502.01850.

Dataset: https://www.kaggle.com/datasets/zhukeyi1/fuji-ripeness-and-size-dataset(GPL-3.0)

[2] Gené-Mola, J. et al. (2021). PFuji-Size dataset: A collection of images and photogrammetry-derived 3D point clouds with ground truth annotations for Fuji apple detection and size estimation in field conditions. Data in Brief, 39, 107629. (CC BY 4.0)

[3] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[4] Nursery Near Me. Apple Gala (Malus domestica), product photo used as test image in Figure 6. https://nurserynearme.com.au/products/apple-gala-malus-domestica (accessed 26 October 2026).

Use of AI tools: Claude (Anthropic) was used to survey available datasets, and for deploying the website, as we were not familiar with Gradio and Render, we also wrote the readme file on Github in Norwegian initially and used AI to translate it to English. Claude was also used to help set up the model selection. It suggested which models from the course to compare, how to compare them fairly (cross-validation grouped by photo, F1 instead of accuracy because of the class imbalance), and how to choose the decision threshold. The choice itself was based on the results: the cross-validation scores from `train.py`, and our own testing of the website on images from outside the dataset. AI was used to generate a first draft of our readme file for Github to help with documentation.